Assignment - 3

Module 3 – Introduction to OOPS Programming

- **Introduction to C++**

1. What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?

⇨ Here, below the difference :-

Procedure Oriented Programming	Object Oriented Programming
It is known as POP	It is known as OOP
It deals with algorithms	It deals with the data
Programs are divided into functions	Programs are divided into objects
Most of the functions share global data	Data structure characterizes objects.
It is top down approach	It is bottom up approach
It needs very less memory	It needs more memory than POP
It do not have any access specifiers.	It has access specifier like private, public and protected
It is less secure	It is more secure
Example :- C, FORTRAN	Example:- C++, JAVA, .NET, PYTHON

2. List and explain the main advantages of OOP over POP.

⇨ There are below the advantages of pop

- **Advantages of OOP :-**
  - ✓ Encapsulation
  - ✓ Polymorphism
  - ✓ Inheritance
  - ✓ Abstraction
  - ✓ Easy Debugging
  - ✓ Modularity
  - ✓ Code Reusability
  - ✓ Better security

3. Explain the steps involved in setting up a C++ development environment.

⇨ There are few steps involved in C++ development environment

- **Install a C++ Compiler :-**
  Install a GCC, MinGW or Visual Studio (for windows).
  The compiler converts the C++ code into a program.

- **Install an IDE or Code Editor :-**
  Use visual studio, code blocks, dev C++ or VS Code to write code easily.

- **Write a C++ Program :-**
  Create a file like main.cpp and write your code.

- **Compile the program**

- **Run the program**

4. What are the main input/output operations in C++? Provide examples.

⇨ There are few main input/output operations

- **Output Operation (cout) :-**
  It is used to display the output on the screen.
  It used to insertion operator <<
  Example :-

```
#include<iostream>
Using namespace std;
Int main()
{
    Cout<<"Hello World";
    Return 0;
}
```

- **Input Operation (cin) :-**
  it is used to take input from the user
  it is used to extraction operator
  Example:-

```
#include<iostream>
Using namespace std;
```

```
Int main()
{
    Int age;
    Cin>>age;
    Cout<<”Age is = ”<<age;
    Return 0;
}
```

- **Error Output :- (cerr)**
  it is used to display a error message.
  Output is unbuffered (shown Immediately)
  Example :-
  cerr << "Error occurred!";

- **Log Output :- (clog)**
  It is used for logging message.
  Output is buffered.
  Example :-
  Clog <<”This is a log message”;

5. What are the different data types available in C++? Explain with examples.

⇨ Basically, there are four types of data types.
- ✓ Primitive Data types
- ✓ Derived Data types
- ✓ User-defined data types
- ✓ Modified Data types

- **Primitive Data types :-** These are the fundamental datatypes built into C++.

Data Type	Description	Example Declaration	Example Usage
Int	Stores whole number	Int a = 10;	Cout<<a;
Float	Stores Decimal Values(single precision)	Float x = 5.5;	Cout<<x;
Double	Stores Decimal Values(double precision)	Double x = 10.2345;	Cout<<x;

char	Stores a Single Character	Char ch = ‘Y’;	Cout<<ch;
Bool	Stores a true or false Value	Bool flag = true;	Cout<<flag;
void	Represents No Value	Void display();	Used in function with no return value

- **Derived Data Types :-**

Data Type	Description	Example Declaration	Example Usage
Array	Collection of same type elements	Int arr[3] = {1,2,3};	Cout<<arr[0];
Pointer	Stores address of variable	Int* ptr = &a;	Cout<<*ptr;
Reference	Alias for variable	Int& ref = a;	Cout<<ref;
Function	Block of code with return type	Int add(int x, int y);	Cout<<(2,3);

- **User-defined Data types :-**

Data Type	Description	Example declaration	Example usage
Struct	Group different data types	Struct student{ Int age; }	Student s1; s1.age = 20;
Class	Blueprints for objects	Class car {public : int speed; }	Car c; c.speed = 100;
Union	Share same memory for different variables	Union data {int a; float b; }	Data d; d.a=5;
enum	Name constants	Enum day {Mon,Tue};	Day d = Mon;
Typedef/using	Creates type alias	Typedef int Marks;	Marks m = 90;

- **Modified data types :-**

Data type	Description	Example declaration	Example Usage
Signed	Allows positive & negative values	Signed int a = -20;	Cout<<a;
Unsigned	Only allows positive values	Unsigned int b = 20;	Cout<<b;
Short	Smaller range integer	Short int x = 100;	Cout<<x;
long	Larger range integer	Long int y = 1000000;	Cout<<y;
Long long	Very large integer	Long long int z = 1234567890;	Cout<<z;

- **Example :-**

```
#include<iostream>
using namespace std;

struct student{
    int age;
};

int add(int a,int b){
    return a + b;
}

int main()
{
    int a = 10; //primitive data type (int=> built in data type)
    int arr[2] = {1,2}; //deroived data type(Array)
    student s;          //user-defined data type
    s.age = 25;

    cout<<"Primitive Data type = "<<a<<endl;
    cout<<"Derived Data type = "<<arr[1]<<endl;
    cout<<"User-defined data type (struct) = "<<s.age<<endl;
    cout<<"Function = "<<add(6,4)<<endl;
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Primitive Data type = 10
Derived Data type = 2
User-defined data type (struct) = 25
Function = 10

--------------------------------
Process exited after 0.3246 seconds with return value 0
Press any key to continue . . .
```

6. Explain the difference between implicit and explicit type conversion in C++.

⇨ There are few differences below :-

Implicit Type Conversion	Explicit Type Conversion
Type conversion is that which automatically converts the one data type to another	Type conversion is that which manually converts the one data type to another
Implemented only when two data types are compatible	Implemented only when two data types are incompatible
No Operator Required	Operator Required
It is done explicitly while compiling	It is done during program designing
It is a widening conversion	It is a narrowing conversion
Example :- Int a = 5, double b = a; → a is automatically converted to double	Example :- double a = 5.7; int b = (int)a; → programmer forces conversion.

7. What are the different types of operators in C++? Provide examples of each.

⇨ There are different types of operators below:-

⇨ **Arithmetic Operator :-**

Operator	Name	Description	Example
+	Addition	Add Two Operands.	5 + 2 = 7
-	Subtraction	Subtract right operands from left	5 – 2 = 3
*	Multiplication	Multiplies by Two Operands	5 * 2 = 10
/	Division	Divides left operands from right	5 / 2 = 2.5

%	Modules	Return the reminder of an integer division	5 % 2 = 0.1

⇨ **Relational Operator :-**

Operator	Name	Description	Example
==	Equal to	Check if two operands are equal.	5 == 2 evaluates to 0 (false)
!=	Not Equal to	Check if two operands are not equal	5 == 2 evaluates to 0 (false)
<	Less than	Checks if left operand is less than right	5 < 2 evaluates to 0 (false)
>	Greater than	Checks if left operand is greater than right	5 > 2 evaluates to 1 (true)
<=	Less than or Equal to	Checks if left operand is less than or equal to	5 <= 2 evaluates to 0 (false)
>=	Greater than or Equal to	Checks if left operand is greater than or equal to	5 >= 2 evaluates to 1 (true)

⇨ **Logical Operator :-**

Operator	Name	Description	Example

&&	Logical AND	Return 1 if both the conditions are true.	(5 > 2) && (2 < 5) evaluates to 1
||	Logical OR	If one condition are true than true otherwise false	Logical OR
!	Logical NOT	Reverses the logical state of an operand	(5 == 2) evaluates to 1

⇨ **Assignment Operator :-**

Operator	Name	Description	Example	Equivalent to
=	Assignment	Assign the value on right to the variable on the left	a = 5	a = 5
+=	Add and Assign	Adds right operand to the left operand and assigns the result to the left.	a+=2	a = a + 2
-=	Subtract and Assign	Subtracts right operand from the left operand and assigns the result to the left.	a-=2	a = a - 2
*=	Multiply and Assign	Multiplies right operand with the left operand and	a*=2	a = a * 2

		assigns the result to the left.		
/=	Divide and Assign	Divides left operand by the right operand and assigns the result to the left.	a/=2	a = a / 2
%=	Modules and Assign	Takes modulus using two operands and assigns the result to the left operand.	a%=2	a = a % 2

⇨ **Increment/Decrement Operators :-**

Operator	Name	Description	Example
++	Increment	Increases the integer value by one.	++a (prefix) or a++ (postfix)
--	Decrement	Decrement the integer value by one.	--a (prefix) or a-- (postfix)

⇨ **Bitwise Operator :-**

Operator	Name	Description
&	Bitwise AND	Sets each bit to 1 if both bits are 1.

|	Bitwise OR	Sets each bit to 1 if either of the bits is 1.
^	Bitwise XOR	Sets each bit to 1 if only one of the bits is 1.
~	Bitwise Complement	Inverts all the bits of the operand.
<<	Left shift	Shifts the bits of the left operand to the left by the number of positions specified by the right operand.
>>	Right shift	Shifts the bits of the left operand to the right by the number of positions specified by the right operand.

⇨ **Conditional Operators :-**

Operator	Name	Syntax	Description	Example
?:	Conditional	condition ? expression1 : expression2	If the condition is true (1), expression1 is executed; otherwise, expression2 is executed.	Result = (marks >=60)? 'pass' : 'fail'

Example :-

Input:-

Data-type.cpp

```cpp
//Data types program in C++
#include<iostream>
using namespace std;

int main()
{
    //Integer Data type
    int age = 25;

    //Float datatype
    float height = 5.7f;
    double salary = 65000.54;

    //character data-type
    char grade = 'A';

    //boolean data-type
    bool isstudent = true;

    //string datatype
    string name = "Yash";

    //long-integer
    long population = 780000000;

    //short-integer
    short year = 2026;

    /*Display the output*/
    cout<<"Integer Value = "<<age<<endl;
    cout<<"Float Value of Height = "<<height<<endl;
    cout<<"Double salary = "<<salary<<endl;
    cout<<"Character of Grade"<<grade<<endl;
    cout<<"Boolean of student = "<<isstudent<<endl;
    cout<<"Name of the student = "<<name<<endl;
    cout<<"Population of World = "<<population<<endl;
    cout<<"Year = "<<year<<endl;

    return 0;
}
```

Output:-

E:\TOPS Tech Documents\Mo

```
Integer Value = 25
Float Value of Height = 5.7
Double salary = 65000.5
Character of GradeA
Boolean of student = 1
Name of the student = Yash
Population of World = 780000000
Year = 2026

--------------------------------
Process exited after 0.1911 seconds with return value 0
Press any key to continue . . .
```

8. Explain the purpose and use of constants and literals in C++.

⇨ **Constant in C++ :-** constant is a variable whose value cannot be changed once it is defined. It is declared using const keyword.

Purpose of Constants :-

- To store a fixed values that should not be changed during program execution.
- To improve code readability.
- To prevent accidental modification of important data.
- To make program easier to maintain.

Example :-

Data-type.cpp | area-of-circle.cpp

```cpp
//const keyword example
#include<iostream>
using namespace std;

int main()
{
    int r;
    const float pi = 3.14;
    float area;

    cout<<"Enter the Radius = ";
    cin>>r;

    area = pi*r*r;

    cout<<"Area of Circle = "<<area;
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Enter the Radius = 6
Area of Circle = 113.04
--------------------------------
Process exited after 5.538 seconds with return value 0
Press any key to continue . . .
```

⇨ **Literals in C++ :-** A literal is a fixed value written directly in the program. It does not change during execution.

Purpose of Literals :-

- To represent fixed value directly in code.
- Used to assign values a variable.
- Used in expression and calculation.

Types of Literals :-

- Integer – literal
- Floating – literal
- Character-literal
- String-literal
- Boolean – literal

Example :-

Int age = 25;

```
Float pi = 3.14;

Char cha = 'A';
```

9. What are conditional statements in C++? Explain the if-else and switch statements.

⇨ Conditional statement are used to control the flow of program based on certain condition.

⇨ They allow the program to make decisions and execute a different blocks of code depending on whether a condition is true or false.

➢ **If-else Statement :-**

✓ Executes one block if the condition is true.

✓ Executes another block if the condition is false.

✓ Example :-

```
If(age>=18){
    cout <<"Eligible for voting";
}else{
    Cout<<"Not Eligible for voting";
}
```

➢ **Switch statement :-**

✓ Used when there are multiple fixed choices for a single variable.

✓ Use Case, break and default.

✓ Example :-

```
Switch(day){
    Case 1:
        Cout<<"Monday";
    Break;
    Case 2:
        Cout<<"Tuesday";
    Break;
    Default:
```

```
            Cout<<"Invalid Day";
        Break;
    }
```

10. What is the difference between for, while, and do-while loops in C++?

⇨ Here's the below the basic difference :-

Feature	While loop	For loop	Do While loop
Condition Check	Checked before each iteration	Checked before each iteration	Checked after each iteration
Minimum Execution	0 times	0 times	At least one time
Loop Structure	Condition only	Initialization, Condition and Modification	Body-first and Condition Last
Risk of infinite loop	Medium	Low	Medium
Typical Syntax Style	While(condition)	For(Initialization; condition; modification)	Do { } while(condition)

11. How are break and continue statements used in loops? Provide examples.

⇨ Here's below the explanation.

- **Break Statement :-**
  - ✓ Used to terminate a loop or switch statement immediately.
  - ✓ Control goes to the statement after the loop or switch.
  - ❖ **Where to used :-** for, while, do..while and switch.
  - ❖ **Example :-**

Input:-

```cpp
//Break Statement in c++
#include<iostream>
using namespace std;

int main()
{
    int i;

    for(i=1;i<=10;i++){
        if(i==5){
            break;
        }
        cout<<i<<endl;
    }
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

1
2
3
4

--------------------------------
Process exited after 0.1707 seconds with return value 0
Press any key to continue . . .
```

➢ **Continue Statement :-**

- ✓ Used to skip current iteration of loop.
- ✓ Loop continues with the next iteration.
  - ❖ **Where to used :-** for, while, do..while and switch.
  - ❖ **Example :-**

Input:-

```cpp
//Continue Statement in c++
#include<iostream>
using namespace std;

int main()
{
    int i;

    for(i=1;i<=10;i++){
        if(i==5){
            continue;
        }
        cout<<i<<endl;
    }
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

1
2
3
4
6
7
8
9
10

--------------------------------
Process exited after 0.1744 seconds with return value 0
Press any key to continue . . .
```

12. Explain nested control structures with an example.

⇨ A nested control structure occurs when a control statement such as , if, if-else, for, while or switch is written inside another control statement .

➢ **Purpose :-**

- ✓ Check Multiple conditions
- ✓ Handle complex-decision making.
- ✓ Control program flow more precisely.

Example :-

Input:-

```cpp
//Nested - loop statement
#include<iostream>
using namespace std;

int main()
{
    int num;

    cout<<"Enter the Number = ";
    cin>>num;

    if(num > 0){
        if(num % 2 == 0){
            cout<<"Number is positive and even number";
        }else{
            cout<<"Number is positive and odd number";
        }
    }else{
        cout<<"Number is Negative";
    }
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Enter the Number = 7
Number is positive and odd number
--------------------------------
Process exited after 1.509 seconds with return value 0
Press any key to continue . . .
```

13. What is a function in C++? Explain the concept of function declaration, definition, and calling.

- ⇨ A functions is a block of code that perform a specific tasks.
- ⇨ It helps divide the big problems into smaller parts.
- ⇨ Every C++ program has at least one function – main().

➢ **Function Declaration :-**

- ✓ Tells the compiler the function is exist.
- ✓ Specifies
- ✓ Return type
- ✓ Function name
- ✓ Parameters written before main()

Syntax :- return type function_name(p1,p2,p3);

Example:- int add(int,int);

- **Function Definition :-**
  - ✓ Contain actual code of the function.
  - ✓ Written after main() or before it.

Syntax :-

```
Return type function_name(parameters){

        //code

}
```

Example :-

```
Int add(int a, int b){

        Return a + b;

}
```

- **Function call :-**
  - ✓ Used to execute the function
  - ✓ Can be called from main() or another function

Syntax:-

```
Function_name(arguments);
```

Example:-

```
Sum = Add(10,20);
```

Program Example :-

Input:-

```cpp
//function in c++

#include<iostream>
using namespace std;

int mul(int a,int b){
    return a * b;
}

int main()
{
    int a,b;

    cout<<"\nEnter the Value of A = ";
    cin>>a;

    cout<<"\nEnter the Value of B = ";
    cin>>b;

    cout<<"\nMultiplication of Two Value = "<<mul(a,b)<<endl;
    return 0;
}
```

Output :-

```
E:\TOPS Tech Documents\Mo

Enter the Value of A = 24

Enter the Value of B = 4

Multiplication of Two Value = 96

--------------------------------
Process exited after 3.413 seconds with return value 0
Press any key to continue . . .
```

14. What is the scope of variables in C++? Differentiate between local and global scope.

⇨ Scope is a part of the program where a defined variable can have it's existence and beyond that it cannot exist that is know as scope.

There are below basic difference :-

Local Variable Scope	Global Variable Scope
Local variable are declared inside the function.	Global variable are declared before the main function.
Local variable cannot be accessed outside the function	Global variable can be accessed in any function.
Local variable are alive only for a function	Global variable are alive till the end of the program.
More reliable and secure since other functions cannot change the value.	Accessible by multiple functions, therefore it's value can be changed.

15. Explain recursion in C++ with an example.

⇨ **Recursion :-**

Recursion is when a function call it's self to solve a problem.
It breaks big problems into smaller.

There are two types of recursion.
Base class :- stops the recursion
Recursive call :- the function call it's self.

Example :-
Input:-

```cpp
/*
    Factorial Calculation Using Recursion
    Aim :- Write a C++ program that calculates the factorial of a number using recursion.
o Objective: Understand recursion in functions.
*/

#include<iostream>
using namespace std;

int fact(int n){
    if(n==1){
        return 1;
    }
    int f;
    f = n*fact(n-1);
    return f;
}

int main()
{
    cout<<"Factorial of Given Number = "<<fact(5)<<endl;
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Factorial of Given Number = 120

--------------------------------
Process exited after 0.1799 seconds with return value 0
Press any key to continue . . .
```

16. What are function prototypes in C++? Why are they used?

⇨ Function prototype is a function declaration that tells the compiler about the function name, return type and parameter before it is used.

Syntax :- return type function_name(parameter);

Example :-

Input:-

```cpp
#include<iostream>
using namespace std;

//function prototype
int add(int,int);

int main()
{
    int result = add(20,30);
    cout<<"Addition is = "<<result;
    return 0;
}
//function declaration
int add(int a,int b){
    return a + b;
}

```

Output:-

```
E:\TOPS Tech Documents\Mo

Addition is = 50
--------------------------------
Process exited after 0.1961 seconds with return value 0
Press any key to continue . . .
```

17. What are arrays in C++? Explain the difference between single-dimensional and multi- dimensional arrays.

- ⇨ Array is used to store multiple collection of same data type.
- ⇨ Each element can be accessed using an index number, starting from 0.
- ⇨ **Example :-** int a[5] = {10,20,30,40};
- ⇨ **Below the difference :-**

Single dimensional array	**Multi dimensional array**
Store element in a single list	Stores elements in a row and column.
Uses one index	Uses two or more indices
Linear structure	Table/matrix structure
Access :- a[i];	Access:- a[i][j];
Example :- a[5];	Example :- int a[3][4];

18. Explain string handling in C++ with examples.

- ⇨ String handling means working with sequence of characters – creating, modifying, comparing, searching and manipulating text.
- ⇨ There are handling in two ways :-
  - ➢ C- Style string (character array)
  - ➢ C++ string class from C++ standard library.

❖ C- style string (character array) :-

These are array of character ending with the null character \0.

Example :-

Input :-

C-style-string.cpp

```cpp
//C-style string example
#include<iostream>
using namespace std;

int main()
{
    char str[20];

    cout<<"Enter the string = ";
    cin>>str;

    cout<<"String is = "<<str<<endl;
    return 0;
}
```

Output :-

```
E:\TOPS Tech Documents\Mo

Enter the string = yash
String is = yash

--------------------------------
Process exited after 2.175 seconds with return value 0
Press any key to continue . . .
```

❖ C++ String class :- string class is easier and safer than C-style strings.

Example :-

Input :-

```cpp
//string-class
#include<iostream>
#include<string>
using namespace std;

int main()
{
    string str = "Hello";
    cout<<str;
    return 0;
}
```

Output :-

```
E:\TOPS Tech Documents\Mo

Hello
--------------------------------
Process exited after 0.2906 seconds with return value 0
Press any key to continue . . .
```

19. How are arrays initialized in C++? Provide examples of both 1D and 2D arrays.

⇨ **Initialized of 1D array :-** 1D array stores in a single row.

Syntax :- datatype array_name[size] = {value};

Example :-

```
#include<iostream>
Using namespace std;
Int main()
{
    Int arr[5] = {10,20,30,40,50};
    for(int i=0;i<=5;i++){
        cout<<arr[i]<<" ";
    }
}
```

Output :-

10,20,30,40,50;

⇨ **Initialized of 2D array :-** A 2D array stores elements in row and column in a matrix form.

Syntax :- datatype array_name[row][col] = {{values}};

Example :-

```
#include<iostream>
Using namespace std;
Int main()
{
    Int matrix[2][3] = {
        {1,2,3},
        {4,5,6}
    };
    for(int i=0;i<2;i++){
      for(int j=0;j<3;j++){
          cout<<matrix[i][j]<<" ";
      }
      Cout<<endl;
    }
}
```

Output :-

1 2 3

4 5 6

20. Explain string operations and functions in C++.

⇨ There are below the string operations and functions :-

Common string operations :-

1. Concatenation(joining string)
2. Finding length
3. Accessing characters
4. Comparing strings.

➢ Concatenation (joining string) :-

Input :-

concatenations(string-operation).cpp

```cpp
#include<iostream>
using namespace std;

int main()
{
    string a,b,c;

    cout<<"Enter the string of a = ";
    cin>>a;

    cout<<"Enter the string of b = ";
    cin>>b;

    c = a + b;
    cout<<c;

    return 0;
}
```

Output :-

```
E:\TOPS Tech Documents\Mo

Enter the string of a = Yash
Enter the string of b = Tailor
YashTailor
--------------------------------
Process exited after 8.171 seconds with return value 0
Press any key to continue . . .
```

- Finding Length :-

Input :-

```cpp
//Finding Length in c++
#include<iostream>
using namespace std;

int main()
{
    string s;
    cout<<"Enter the string = ";
    cin>>s;

    cout<<s.length();
    return 0;
}
```

Output :-

```
E:\TOPS Tech Documents\Mo

Enter the string = yash
4
--------------------------------
Process exited after 3.065 seconds with return value 0
Press any key to continue . . .
```

- Accessing Characters :-

  Input :-

```cpp
//Accessing characters
#include<iostream>
using namespace std;

int main()
{
    string s;
    cout<<"Enter the Name = ";
    cin>>s;

    cout<<"Find Character Number = "<<s[3]<<endl;
    return 0;
}
```

  Output :-

```
E:\TOPS Tech Documents\Mo

Enter the Name = Yash
Find Character Number = h

--------------------------------
Process exited after 12.18 seconds with return value 0
Press any key to continue . . .
```

- Comparing strings :-

  Input :-

```cpp
//compare strings
#include<iostream>
using namespace std;

int main()
{
    string a,b;

    cout<<"Enter the string of a = ";
    cin>>a;

    cout<<"Enter the string of b = ";
    cin>>b;

    if(a==b){
        cout<<"Equal a string!..";
    }else{
        cout<<"Not Equal a string!...";
    }
    return 0;
}
```

Output :-

```
E:\TOPS Tech Documents\Mo
Enter the string of a = apple
Enter the string of b = apple
Equal a string!..
--------------------------------
Process exited after 8.48 seconds with return value 0
Press any key to continue . . .
```

⇨ **Common string functions :-**

➢ **Length() or size :-** return length of the string

```
cout<<s.length();
```

➢ **Substr() :-** return part of a string.

```
string s = "programming";
cout<<s.substr(3,4);
```

output :- gram

➢ Append() :- add another string

```
String a = "Hello";
a.append(" world");
```

➢ Insert() :- inserts characters at a position

```
String s = "Hello World";
s.insert(5, " ");
```

➢ Erase() :- removes characters

```
String s = "HelloWorld";
s.erase(5,5);
```

21. Explain the key concepts of Object-Oriented Programming (OOP).

⇨ There are several key concept of oop

➢ **Class :-** class is a blueprint used to creating an object. It defines variable and functions.

Example :-

```
class car{
```

```
        public:
            string brand;
            int speed;
           cout<<"Car is driving"<<endl;
}
```

- **Object :-** an object is an instance of an class. It represent a real-world thing and uses the class properties and functions.

  Example :-

  ```
  Car c1;
  c1.brand = "Toyota";
  c1.speed = 130;
  ```

- **Encapsulation :-** encapsulation means wrapping the data and functions together into a single unit that is known as encapsulation. Some component using access specifiers like public, private and protected.

  Example :-

  ```
  Class BankAccount{
          Private:
             Double balance;
          Public:
             Void deposit(double amount){
                    Deposit+=amount;
             }
  };
  ```

- **Inheritance :-** inheritance allows one class inherits and properties to the another class that is known as inheritance.

  Example :-

  ```
  Class Animal{
          Public:
            Void eat(){
                    Cout<<"Animal Eats"<<endl;
            }
  }
  Class Dog : Animal{
          Public:
             Void Bark(){
  ```

```
            Cout<<"Dogs are barking"<<endl;
        }
}
```

- **Polymorphism :-** polymorphism means one name and many forms that is known as polymorphism.

  There are two types of polymorphism :-

  1. Compile time polymorphism. (function overloading / function overriding)
  2. Run time polymorphism.(function overloading)

  Example :- (function overloading)

```
Class Math{
        Public:
          Int add(int a, int b){
                Return a + b;
          }
          Int add(int a, int b, int c){
                Return a + b + c;
          }
}
```

  Note :- add() functions works with different parameters.

- **Abstraction :-** Abstraction means to hide certain details and only show essential information.

  Example :-

```
class car{
    public:
        void start(){
            cout<<"Car started!..."<<endl;
        }
}
```

22. What are classes and objects in C++? Provide an example.

- **Classes :-** class is a blueprint template used to creating an object.

a class car can contain data and functions
Data :- color, speed
Function :- start(), stop().

⇨ **Objects :-** objects is an instance of an class.
if car is a class then car1, car2 is an object.

Example :-

Input :-

class&object.cpp

```cpp
//class and object examples in c++
#include<iostream>
using namespace std;

class car{
    public:
        string brand;
        int speed;

        void display(){
            cout<<"Brand = "<<brand<<endl;
            cout<<"Speed = "<<speed<<endl;
        }
};

int main()
{
    //creating an object
    car c1;

    c1.brand = "Agera R";
    c1.speed = 300;

    c1.display();
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Brand = Agera R
Speed = 300

--------------------------------
Process exited after 0.1843 seconds with return value 0
Press any key to continue . . .
```

23. What is encapsulation in C++? How is it achieved in classes?

⇨ Encapsulation means wrapping data and method together in a single unit that is known as encapsulation.

This helps :
Protect data
Control access
Improve security of program

Encapsulation Achieved using
Classes
Access specifier

Access specifier :-
Private :- Accessible only inside the class.
Public :- Accessible from outside the class.
Protected :- Accessible class and derived class.

24. What is inheritance in C++? Explain with an example.

⇨ Where one class(derived / child class) acquires the properties and behaviors of the another class that is known as Inheritance.

Syntax :- class derived class : access_specifier Base_class{

};

Example :-

Input:-

```cpp
/*
    Aim :- Write a program that implements inheritance using a base class Person and derived
classes Student and Teacher. Demonstrate reusability through inheritance.
*/
#include<iostream>
using namespace std;

//base class
class Person{
    public:
        string name;
        int age;

        void getPerson(){
            cout<<"Enter Name = ";
            cin>>name;
            cout<<"Enter Age = ";
            cin>>age;
        }

        void showPerson(){

            cout<<"Name = "<< name << endl;
            cout<<"Age = "<< age << endl;
        }
};
```

Output:-

```
===============Student's Details=================
Enter Name = Yash
Enter Age = 25
Enter Marks = 89

Name = Yash
Age = 25
Marks = 89
```